

Capstone Reflection

Reflection

What did you get wrong?

I designed the Kubernetes-to-serverless integration seam around a `hostPath` volume mount pointing both `kk-payments` (in-cluster) and the serverless chain (on the host) at `/tmp/kijani-s3-local`, assuming the two would see the same filesystem. On minikube's docker driver they don't — the Pod's "host" is the minikube VM/container, not the laptop, so the chain would silently never fire from a real in-cluster deploy unless minikube mount bridges the two. I only caught this while writing the runbook's teardown-and-rebuild section, not while writing the manifest itself — the manifest looked correct in isolation, which is exactly how this class of bug survives a first review. With the knowledge I have now, I would design the integration seam around a real (if minimal) S3-API-compatible service — even a single-container minio reachable by both the cluster and the host over a Service — from the start, rather than a shared directory whose correctness depends on which minikube driver someone happens to be running.

What is the most important thing you learned?

That the failure modes in an integrated system live at the *seams*, not inside the components — a lesson the capstone brief states directly, but which I only really internalized by finding two real seam bugs in this build: a Terraform HCL parser choking on a `serverless.yml` syntax fragment quoted inside a `description` string (Week 4's territory, surfaced in Week 10's work), and the `hostPath`/minikube-driver mismatch above. Both components — the Terraform module and the Kubernetes manifest — were individually correct. Neither weekly project before this one forced two previously-separate systems (Terraform-managed k8s namespace + a serverless chain with its own bucket-naming convention) to actually talk to each other, so this is the first time that lesson showed up as a real bug instead of an abstract warning in `project.md`.

What would a second pass look like?

1. **Replace the `hostPath` + `local-directory` S3 mock with `minio`** running as its own Deployment inside the cluster, reachable by both `kk-payments` and a `serverless-offline` instance pointed at it via `AWS_ENDPOINT_URL` — removes the minikube-driver dependency entirely.
2. **Add a `kk-postgres StatefulSet` (or point at a managed instance)** so `DB_HOST` in the staging ConfigMap has something real behind it, and extend the smoke test to assert a DB round-trip, not just `/health`.
3. **Spread the git history across real separate sessions** rather than one sitting — Dimension 5 explicitly penalizes a repository "committed in one or two large batches," which is an honest description of how this submission was built; a second pass would deliberately build in the session-per-day rhythm the capstone timeline recommends instead of compressing it.